

FLOWFORGE

Project Scope & Engineering Blueprint

A production-style multi-tenant workflow automation platform

Zero-code starting point

Architecture-first development

Owner-controlled AI decisions

Prepared as the source brief for implementation with VS Code Codex

1. Executive Summary

FlowForge is a multi-tenant SaaS platform that lets organizations design, publish, execute, monitor, approve, and audit internal business workflows. The platform is intentionally designed as more than a CRUD dashboard: it demonstrates durable workflow execution, human approval, asynchronous processing, concurrency control, tenant isolation, realtime updates, auditability, reliability, and measured scalability.

Canonical example:

Manual Trigger -> Manager Approval -> Amount > 5000? -> Finance Approval -> Email -> Accounting Webhook
-> End

The project starts from zero application code. The implementation will be incremental and milestone-based. The coding assistant may recommend options, but major architecture, security, data-model, and scope decisions remain under owner approval.

Project Goals

Goal	Target
Product	Internal workflow automation and approval platform
Initial concurrency	~100 concurrent users
Target concurrency	~1,000 concurrent users
Architecture	Modular monolith with scalable workers
Reliability	Durable state, retries, idempotency, outbox
Security	Multi-tenant isolation, RBAC, secure integrations
Delivery	Incremental milestones with tests before progression

2. Product Scope

Identity & Organization Management

- Registration, login, logout, verification, password reset
- Multiple sessions/devices
- Organizations and organization switching
- Invitations and memberships
- Teams, roles, and permissions

Workflow Design

- Create and edit workflow drafts
- Node/edge visual builder
- Validation before publication
- Immutable published versions

- Version history

Workflow Execution

- Asynchronous execution
- Persistent execution and step state
- Human approval pause/resume
- Condition branching
- Email and webhook steps
- Delay/scheduling
- Retries and cancellation

Operational Experience

- Task inbox
- Execution timeline
- Realtime updates
- Notifications
- Files and attachments
- Audit history
- Basic analytics/dashboard
- API keys and webhook integrations

Initial User Roles

- Platform Super Admin
- Organization Owner
- Organization Admin
- Workflow Designer
- Approver
- Member/Employee
- Viewer/Auditor

3. Scope Boundaries

The first version deliberately avoids features that would add large amounts of complexity before the core workflow engine is correct.

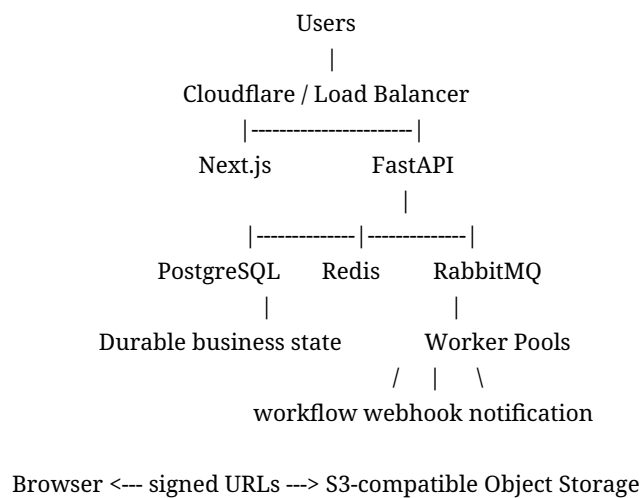
- Arbitrary custom-code/script execution
- General loops
- Complex parallel fork/join
- Full BPMN compatibility
- Hundreds of third-party integrations
- Enterprise SSO and billing
- Marketplace

- Cross-region active-active deployment
- Kafka, Kubernetes, or microservices by default
- AI-generated workflows

These are future options, not initial requirements. New major capabilities require an explicit owner decision.

4. High-Level Architecture

FlowForge starts as a modular monolith. The frontend, API, workers, and supporting infrastructure have clear responsibilities, while critical business state remains centralized in PostgreSQL.



Architecture Principles

- PostgreSQL is the source of truth for critical state.
- Redis is for cache, rate limits, and realtime fan-out; it is not the business database.
- RabbitMQ transports asynchronous jobs; it is not the workflow database.
- Long-running work does not stay inside HTTP requests.
- Workers can scale independently of API servers.
- Major services can be extracted later only when justified by real load or ownership needs.

5. Technology Baseline

Area	Technology	Purpose
Frontend	Next.js App Router + React + TypeScript	Large authenticated SaaS UI and workflow builder
UI	Tailwind CSS + shadcn/ui	Consistent design system
Server state	TanStack Query	Fetching, cache, invalidation, pagination
Complex client state	Zustand	Workflow-builder state
Backend	FastAPI + Python	Typed API and domain services

ORM	SQLAlchemy 2.x + Alembic	Persistence and migrations
Database	PostgreSQL	Transactional source of truth
Cache/realtime	Redis	Cache, rate limits, Pub/Sub
Background work	Celery	Worker execution
Broker	RabbitMQ	Reliable asynchronous task transport
Files	S3/R2/MinIO	Object storage
Realtime	WebSockets	Live execution/task/notification updates
Observability	structlog + OpenTelemetry	Logs, metrics, traces

6. Data Model & Multi-Tenancy

A user can belong to multiple organizations. Membership, not the user row itself, defines the tenant relationship and role context.

- Tenant-owned records generally carry `organization_id`.
- Queries must scope resource ID and organization ID together.
- Published workflow versions are immutable.
- Executions reference the exact workflow version they started with.
- Critical invariants use database constraints in addition to application validation.
- JSONB is used only for legitimately flexible configuration, such as node settings.

Core data groups:

- Identity: users, sessions
- Tenancy: organizations, memberships, invitations, roles, permissions, teams
- Workflow definitions: workflows, versions, nodes, edges
- Runtime: executions, steps, attempts, approval tasks, execution events
- Platform: notifications, files, API keys, webhooks
- Reliability/compliance: outbox events, audit events

7. Workflow Engine Semantics

The engine is a persistent state machine. Execution progress is stored durably so that a process restart or worker crash cannot erase the workflow state.

- Initial execution states: PENDING, QUEUED, RUNNING, WAITING, COMPLETED, FAILED, CANCELLED.
- Jobs are processed at step-level rather than keeping one worker alive for the whole workflow.
- Human approval creates a task, sets the step/execution to WAITING, and releases the worker.
- Resume actions create durable continuation work.
- Retries are recorded as attempts with backoff and jitter.

- Delay nodes persist resume_at rather than sleeping workers.
- At-least-once delivery is assumed; workers must be idempotent.
- Transactional outbox prevents accepted work from being lost when the message broker is unavailable.
- External exactly-once side effects are not promised unless the external provider supports safe idempotency.

8. Security Scope

Authentication

Secure browser sessions or short-lived access plus revocable refresh sessions; Argon2id password hashing.

Tenant isolation

Every tenant-owned lookup is organization-scoped. Knowing another tenant's UUID never grants access.

Authorization

Permission-based RBAC plus resource-level rules such as task assignment.

Webhooks

HMAC signatures, timestamp/replay controls, rate limits and payload limits.

Outbound requests

SSRF protection blocks localhost/private/link-local/internal metadata targets.

Files

Private object storage, short-lived signed URLs, tenant authorization and upload validation.

Secrets

No plaintext secrets in logs or source control; integration credentials require protected storage.

Audit

Important security and business actions are append-oriented and attributable.

9. Non-Functional Targets

Concern	Target / Principle
Concurrency	~100 initial; ~1,000 target
API latency	Typical simple P95 ~<300 ms where practical
Complex lists	P95 ~<500 ms target
Realtime	~1-2 s propagation target

Availability	~99.9% design target
Consistency	Strong for approvals/permissions/state; eventual for analytics
Recovery	Backups/PITR and tested restoration
Scale	Horizontal API and worker scaling
Testing	Correctness + concurrency + failure + load tests before scalability claims

10. Implementation Approach

The system will be built one milestone at a time. Codex must plan first, request owner approval for major decisions, implement the approved milestone, run tests/checks, report results, update documentation, and stop until asked to continue.

1. Repository/local infrastructure bootstrap
2. Authentication and sessions
3. Organizations and tenancy
4. RBAC and teams
5. Workflow definitions/versioning
6. Workflow builder
7. Execution persistence and core engine
8. Approval tasks
9. RabbitMQ/Celery/outbox
10. Retries/recovery/scheduler
11. Realtime
12. Notifications
13. Files
14. Webhooks
15. Audit and analytics
16. Security hardening
17. Integration/E2E testing
18. Load testing and optimization
19. Production deployment

11. Owner-Controlled AI Decision Policy

AI is an engineering assistant, not the final authority. It may autonomously make low-risk implementation details that remain inside the approved architecture. It must stop and ask the owner before materially changing architecture, core technology, tenancy, security, data-model direction, workflow semantics, infrastructure strategy, major dependencies, scope, or destructive migrations.

Required decision format:

Decision required -> Recommended option -> Why -> Alternatives -> Tradeoffs -> Waiting for owner approval

12. Documentation Package

The accompanying Markdown files inside /docs are the implementation source of truth for Codex. The root CODEX_MASTER_PROMPT.md tells Codex how to read them, how to propose the first plan, when it must ask for approval, and when it is allowed to proceed.

- Project scope
- System architecture
- Frontend
- Backend
- Database
- Workflow engine
- API and realtime
- Security
- DevOps and deployment
- Testing and performance
- Implementation rules
- Initial roadmap

End of project scope brief.